Projeto SA - Documentação Técnica - DS V01
01/12/25

Alunos **(Desenvolvimento de Sistema)**: Guilherme Silveira, Caetano Petry, Henrique Nau, Eduardo Ribeiro.
Alunos **(ADM)**: Yasmim Santos, Isabel Cortez, Arthur Gomez, Maria Eduarda.

1 - O que é a GameChip ?

- **Somos mais do que um e-commerce; somos o elo entre a nostalgia do passado e a conveniência do futuro. Cada transação e cada cliente satisfeito é a prova de que a visão do pai de um de nossos sócios está mais viva do que nunca. Nossa missão é ser a ponte digital, oferecendo uma plataforma intuitiva, segura e com atendimento premium.**

1.1 - Tecnologias Utilizadas:

- ☐ **Frontend - (HTML, CSS);**
- ☐ **Backend - (Python, JavaScript);**
- ☐ **FrameWorks - (Flask);**
- ☐ **Banco de Dados - (MySQL).**

2 - Requisitos Funcionais:

Requisitos Funcionais - RF

RF01 – Cadastro de Cliente
O sistema deve permitir o cadastro de clientes com nome, e-mail, CPF, senha e dados pessoais.
RF02 – Login de Cliente
O sistema deve permitir que clientes realizem login usando e-mail e senha.
RF03 – Cadastro de Empresa (Desenvolvedores/Vendedores)
O sistema deve permitir o cadastro de empresas com CNPJ, razão social, fantasia, endereço e senha.
RF04 – Login de Empresa
O sistema deve permitir que empresas acessem o painel para gerenciar seus produtos.
RF05 – Login Administrativo
Funcionários devem acessar o painel administrativo com e-mail e senha.
RF06 – Listagem de Produtos
O sistema deve listar todos os jogos/produtos disponíveis, com imagens, preços e desconto quando houver.
RF07 – Buscar Produtos
O usuário deve conseguir pesquisar produtos por nome, descrição, marca ou categoria.
RF08 – Exibição de Detalhes do Produto
O sistema deve mostrar detalhes de cada produto: nome, descrição, preço, imagens e avaliações.
RF09 – Carrinho de Compras
O usuário deve adicionar, remover e atualizar itens no carrinho.
RF10 – Finalizar Compra
O usuário deve finalizar o carrinho escolhendo método de pagamento.
RF11 – Pagamento por PIX
O sistema deve gerar QR Code e código copia-e-cola PIX automaticamente.
RF12 – Registrar Pedidos
Ao finalizar a compra, o sistema deve registrar o pedido com seus itens e valores.
RF13 – Atualizar Estoque
O sistema deve reduzir o estoque de um produto após a compra.
RF14 – Gerenciar Endereços
O usuário pode cadastrar, editar, excluir e definir endereços principais.
RF15 – Avaliar Produto
Clientes e empresas podem avaliar produtos com nota, título e comentário.

RF16 – Avaliar Empresas
Clientes e empresas podem avaliar outras empresas (exceto si mesmas).
RF17 – Painel da Empresa
Empresas podem gerenciar seus produtos, estoque e preços.
RF18 – Adicionar Produto à Loja da Empresa
Empresas podem registrar que vendem determinado produto, definindo preço e disponibilidade.
RF19 – Remover Produto
Empresas podem remover produtos de sua loja.
RF20 – Atualizar Produto
Empresas podem atualizar preço, estoque e disponibilidade.
RF21 – Sistema de Ofertas
O sistema deve mostrar produtos com preços promocionais na página inicial.
RF22 – Recuperação de Senha
O sistema deve permitir solicitar recuperação por e-mail.

2.1 - Requisitos Não Funcionais:

Requisitos Não Funcionais - RNF

RNF01 – Linguagem e Framework
O sistema deve ser desenvolvido em Python com Flask.
RNF02 – Banco de Dados
O sistema deve usar MySQL.
RNF03 – Segurança
Senhas devem ser armazenadas usando hash seguro (Werkzeug).
RNF04 – Sessões Seguras
O sistema deve usar tokens e sessão com chave secreta.
RNF05 – Validação de Entrada
CPF, CNPJ, email e telefone devem ser validados pelo backend.
RNF06 – Limite de Upload
Imagens dos produtos têm limite de 16MB.
RNF07 – Suporte a Múltiplas Imagens
Produtos devem aceitar imagens no formato JSON.
RNF08 – Responsividade
O frontend deve funcionar em dispositivos móveis e desktop.
RNF09 – Tempo Máximo de Sessão
Sessões expiram após 1 hora de inatividade.
RNF10 – Padronização Visual
Interface deve seguir padrão minimalista no estilo Steam/Games.
RNF11 – Disponibilidade
O sistema deve permanecer disponível durante horário de uso escolar.
RNF12 – Performance
Carregamento de páginas deve levar menos de 2 segundos (ambiente local).
RNF13 – Logs
Ações críticas devem gerar logs básicos (ex: login admin).
RNF14 – Integração com API de Pagamento
QR code PIX deve ser gerado de forma programática.
RNF15 – Tratamento de Erros
O sistema deve exibir mensagens amigáveis em caso de falha.

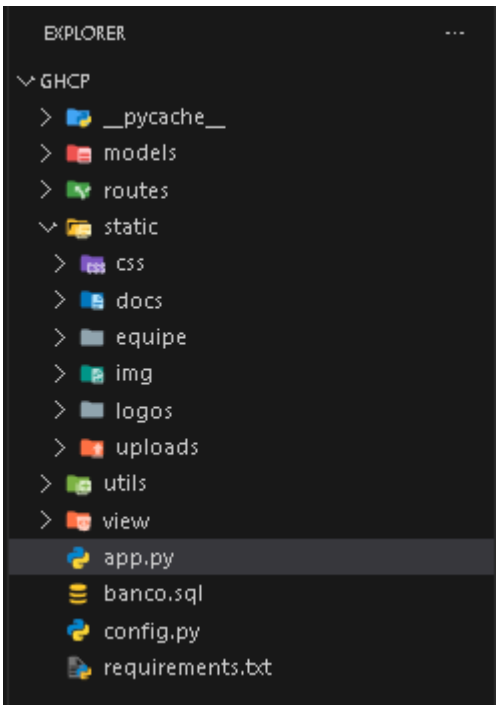
3 - Arquitetura MVC:

O sistema GameChip utiliza o padrão arquitetural MVC (Model–View–Controller) para separar as camadas de lógica, apresentação e acesso aos dados.

- **Model: responsável pelas regras de negócio e comunicação com o banco de dados**

- **View: responsável pela apresentação visual ao usuário**

- **Controller: responsável por receber as requisições, processar os dados e retornar respostas**

1. 

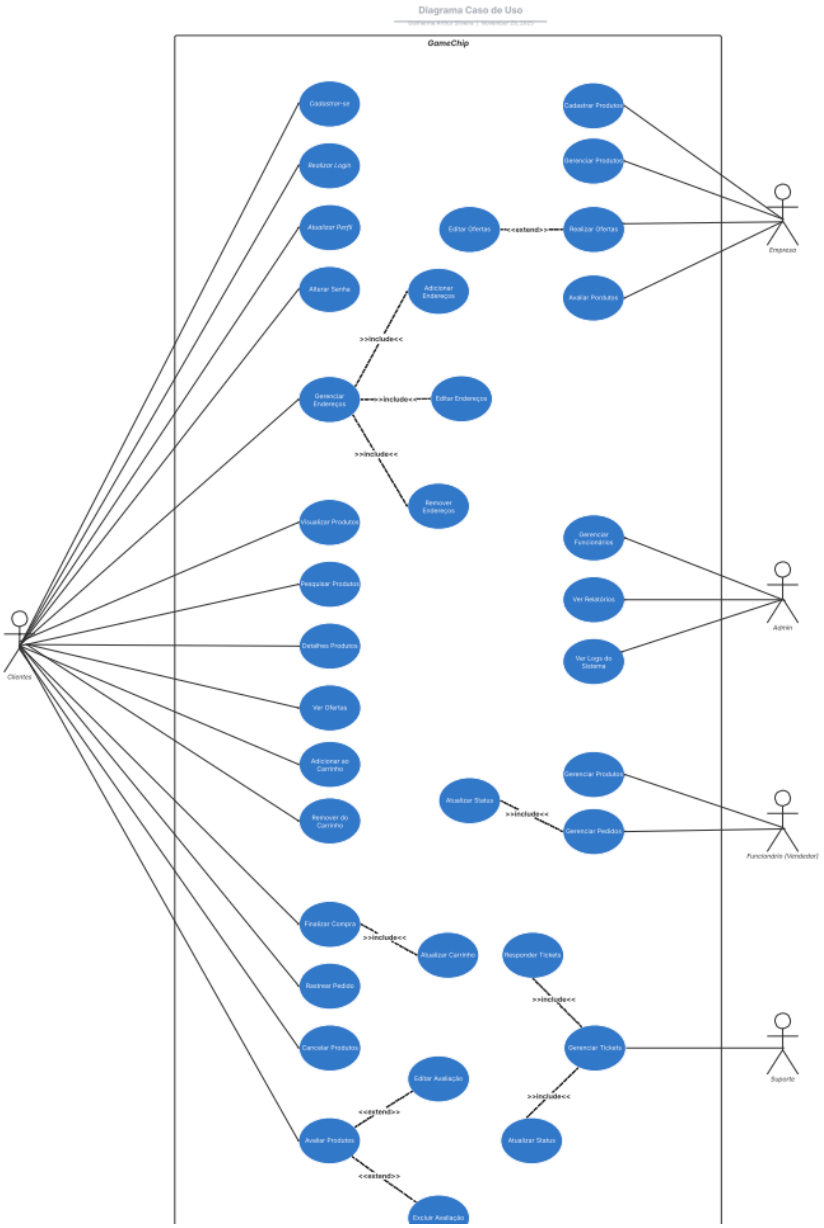
4 - Diagramas:

4.1 - Casos de uso (UML)

Níveis:

- **Usuário faz login**

- **Usuário gerencia cadastro**

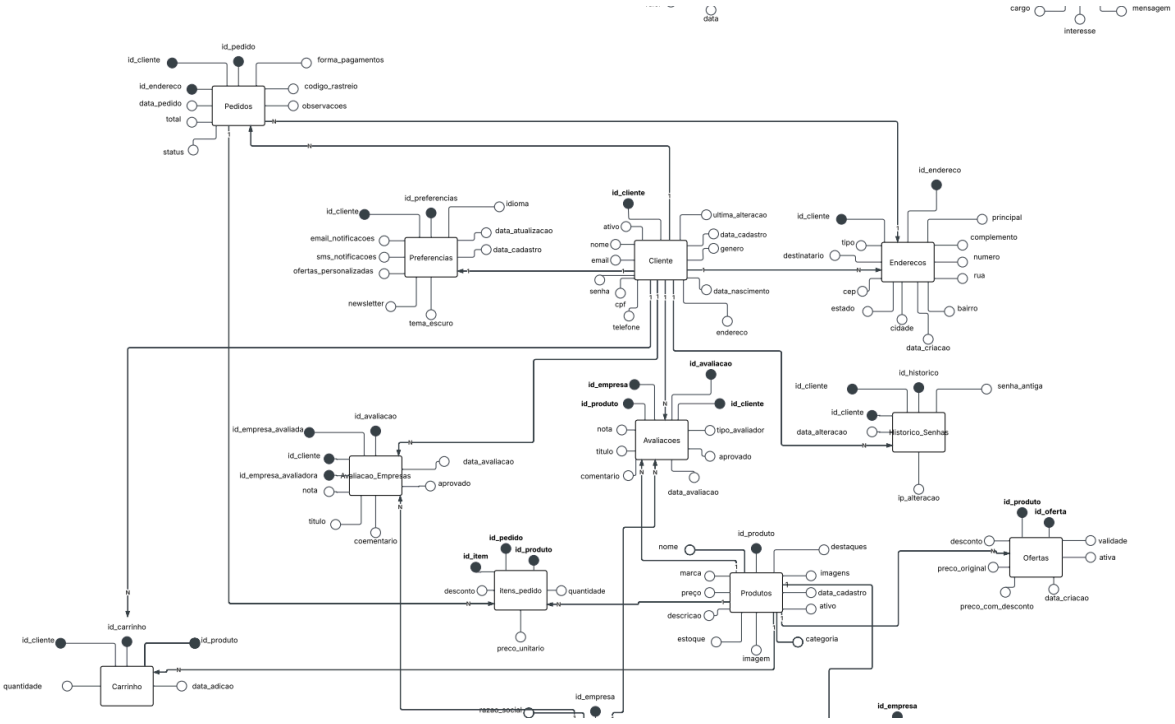


4.2 - DER – Diagrama Entidade Relacionamento

Mostra:

- **tabelas**
- **PK**
- **FK**

- **cardinalidades (1:N, N:M)**



5 - Banco de Dados

‘Nosso time decidiu optar pelo Workbench MySQL, pois é um banco de dados com o qual temos mais conhecimentos e facilidade de usar.’

5.1 - Tabelas Principais:

Cliente:

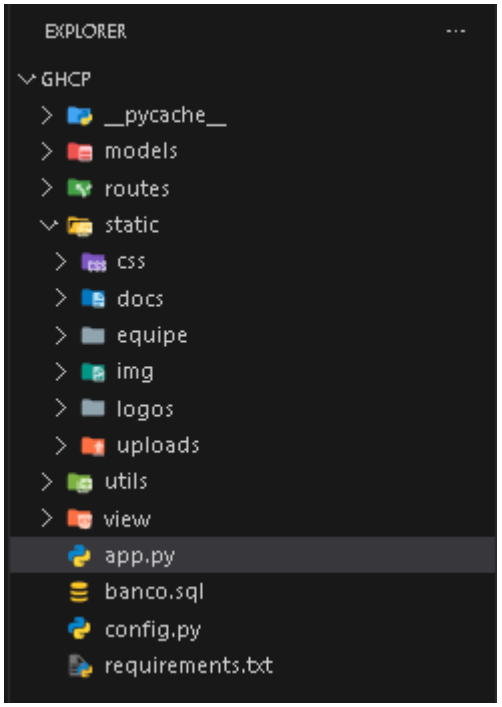
- id_cliente
- nome
- email
- senha

- cpf
- telefone
- data_nasc
- endereco
- genero
- data_cadastro
- ultima_altercao

Produtos:

- id_produto
- nome
- marca
- preco
- descricao
- estoque
- imagem
- categoria
- ativo
- data_cadastro
- imagens
- destaques

6 - Backend:



- **Models** - Armazena todas as classes e funções responsáveis pela regra de negócio e pela comunicação com o banco de dados.

```python
import mysql.connector
from mysql.connector import Error
from werkzeug.security import generate_password_hash
from config import Config

def get_db_connection():
    try:
        return mysql.connector.connect(**Config.DB_CONFIG)
    except Error as err:
        print(f"Erro ao conectar ao banco de dados: {err}")
        return None

def criar_tabelas_necessarias():
    """Cria tabelas que podem estar faltando no banco de dados"""
    try:
        conn = get_db_connection()
        if not conn:
            print("❌ Erro ao conectar ao banco para criar tabelas")
            return

        cursor = conn.cursor()

        # Tabela de empresas (se não existir)
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS empresas (
                id_empresa INT AUTO_INCREMENT PRIMARY KEY,
                razao_social VARCHAR(255) NOT NULL,
                nome_fantasia VARCHAR(255),
                cnpj VARCHAR(18) UNIQUE NOT NULL,
                email VARCHAR(255) UNIQUE NOT NULL,
                senha VARCHAR(255) NOT NULL,
                telefone VARCHAR(20),
                tipo_empresa ENUM('comprador', 'vendedor', 'ambos') DEFAULT 'comprador',
                endereco TEXT,
                ativo BOOLEAN DEFAULT TRUE,
                data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
            )
        """)
```

- **Routes - Contém os arquivos responsáveis por controlar as rotas do sistema, recebendo requisições e retornando respostas.**

```python
@app.route('/remover-carrinho/<int:id_produto>', methods=['POST'])
def remover_carrinho(id_produto):
    if 'carrinho' in session:
        carrinho = session['carrinho']
        session['carrinho'] = [item for item in carrinho if item['id_produto'] != id_produto]
        session.modified = True
        flash('🗑️ Produto removido do carrinho!', 'success')
    return redirect(url_for('carrinho'))

@app.route('/atualizar-carrinho', methods=['POST'])
def atualizar_carrinho():
    if 'carrinho' in session:
        carrinho = session['carrinho']
        carrinho_dict = {item['id_produto']: item for item in carrinho}
        carrinho_atualizado = []
        for key, value in request.form.items():
            if key.startswith('quantidade_'):
                try:
                    id_produto = int(key.split('_')[1])
                    nova_quantidade = int(value)
                    if id_produto in carrinho_dict:
                        item = carrinho_dict[id_produto]
                        if nova_quantidade > 0:
                            item['quantidade'] = nova_quantidade
                            carrinho_atualizado.append(item)
                except ValueError:
                    continue
        session['carrinho'] = carrinho_atualizado
        session.modified = True
        flash('✅ Carrinho atualizado!', 'success')
    return redirect(url_for('carrinho'))

@app.route('/limpar-carrinho', methods=['POST'])
def limpar_carrinho():
    session.pop('carrinho', None)
    flash('🗑️ Carrinho limpo!', 'success')
    return redirect(url_for('carrinho'))
```

- **App.py - É o ponto de entrada do sistema.**

```python
from flask import Flask
from config import Config
from routes.main_routes import configure_main_routes
from routes.auth_routes import configure_auth_routes
from routes.empresa_routes import configure_empresa_routes
from routes.admin_routes import configure_admin_routes
from routes.produto_routes import configure_produto_routes
from routes.carrinho_routes import configure_carrinho_routes
from utils.helpers import from_json_filter
from routes.avaliacao_routes import avaliacao_bp
import os

def create_app():
    app = Flask(__name__, template_folder="view", static_folder="static")
    app.config.from_object(Config)

    app.jinja_env.filters['from_json'] = from_json_filter

    # Configurar rotas
    configure_main_routes(app)
    configure_auth_routes(app)
    configure_empresa_routes(app)
    configure_admin_routes(app)
    configure_produto_routes(app)
    configure_carrinho_routes(app)

    # REGISTRAR BLUEPRINT DAS AVALIAÇÕES ← NOVA LINHA
    app.register_blueprint(avaliacao_bp)

    os.makedirs(Config.UPLOAD_FOLDER, exist_ok=True)

    return app

app = create_app()

if __name__ == '__main__':
    from models.database import criar_tabelas_necessarias, criar_admin_padrao
```

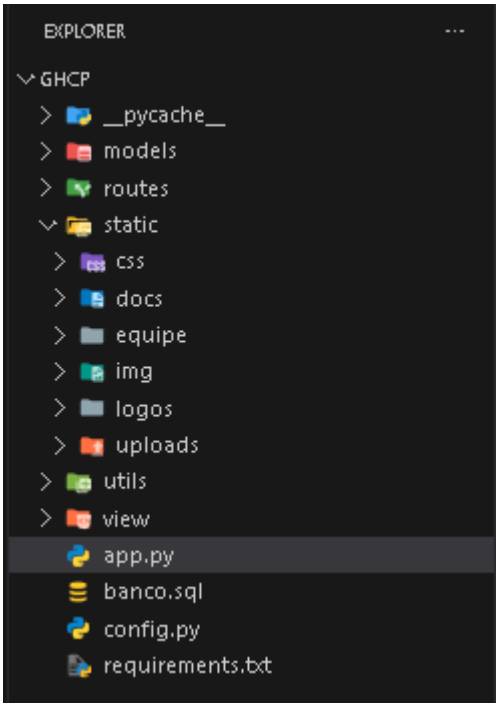
- **Banco SQL - Arquivo que contém toda a estrutura do banco de dados**

```sql
-- Tabela produto
CREATE TABLE produto (
    id_produto INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(255) NOT NULL,
    marca VARCHAR(255) NOT NULL,
    preco DECIMAL(10, 2) NOT NULL,
    descricao TEXT,
    estoque INT DEFAULT 0,
    imagem VARCHAR(500),
    categoria VARCHAR(100),
    ativo BOOLEAN NOT NULL DEFAULT TRUE,
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    imagens JSON,
    destaque BOOLEAN DEFAULT FALSE,
    peso DECIMAL(8,2) DEFAULT 0,
    dimensoes VARCHAR(50),
    INDEX idx_nome (nome),
    INDEX idx_marca (marca),
    INDEX idx_categoria (categoria),
    INDEX idx_ativo (ativo),
    INDEX idx_preco (preco),
    INDEX idx_estoque (estoque)
);

-- Tabela clientes
CREATE TABLE clientes (
    id_cliente INT PRIMARY KEY AUTO_INCREMENT,
    nome VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    cpf VARCHAR(14) NOT NULL UNIQUE,
    telefone VARCHAR(20),
    endereco TEXT,
    data_nascimento DATE,
    genero ENUM('M', 'F', 'O') DEFAULT NULL,
    data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    ultima_alteracao TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    ativo BOOLEAN DEFAULT TRUE,
    INDEX idx_nome (nome),
    INDEX idx_email (email),
    INDEX idx_cpf (cpf),
    INDEX idx_data_cadastro (data_cadastro)
);
```

- **Config.py - Armazena funções e variáveis de configuração como**

```python
import os
from datetime import timedelta

class Config:
    SECRET_KEY = 'GHCP-2o25'

    # Database Configuration
    DB_CONFIG = {
        'host': 'tini.click',
        'port': '3306',
        'user': 'loja_informatica',
        'password': '7787a5c08b46a3ada35c3a1f7ca7dd9b',
        'database': 'loja_informatica'
    }

    # Upload Configuration
    UPLOAD_FOLDER = 'static/uploads/produtos'
    ALLOWED_EXTENSIONS = {'png', 'jpg', 'jpeg', 'gif', 'webp'}
    MAX_CONTENT_LENGTH = 16 * 1024 * 1024  # 16MB

    # Session Configuration
    SESSION_TYPE = 'filesystem'
    PERMANENT_SESSION_LIFETIME = timedelta(hours=1)

    # PIX Configuration
    PIX_CHAVE = "14057629939"
    PIX_NOME = "CAETANO GBUR PETRY"
    PIX_CIDADE = "JOINVILLE"
```

7 - Frontend:



- **Static - Pasta que reúne todos os arquivos estáticos utilizados pelo front-end.**

• css/

Contém todos os arquivos de estilização do site.
Aqui ficam:

- **Arquivos `.css`**
- **Temas de cores**
- **Ajustes de layout**
- **Estilos das páginas e componentes**

Função: definir a aparência visual do site.

• img/

Contém imagens gerais do projeto.
Aqui ficam:

- **imagens de produtos**
- **banners**
- **artes da loja**
- **backgrounds**
- **assets gráficos**

Função: atender às necessidades visuais das páginas.

• logos/

Contém os logotipos da plataforma, variações de cor, ícones e arquivos usados como:

- **favicon**
- **logo no header**
- **logo no login/cadastro**

• uploads/

Pasta onde o sistema guarda imagens enviadas pelos usuários.
Pode incluir:

- **fotos de perfil**
- **imagens de produtos adicionados por admins**

- **View - Contém as Views, que no MVC são as páginas que o usuário realmente vê.**

Aqui ficam:

- **arquivos HTML**
- **templates Jinja2**
- **páginas completas como:**
  - **página inicial**
  - **página de produtos**
  - **carrinho**
  - **login e cadastro**
  - **perfil**
  - **compras**

A pasta "view" é literalmente a interface do usuário.

8 - Versionamento com Github:

8.1 - Commits:

Current repository
GameChip

Current branch
main

Changes | History

Select branch to compare...

Add admin documentação
Djonathan Tamanini • 3 days ago

melhorias de avaliacoes (AINDA POSSU...
Caetano Petry • 3 days ago

Merge branch 'antes-da-api-asaas' of ht...
Caetano Petry • 3 days ago

Tirando o botão de avaliar da página de ...
Caetano Petry • 3 days ago

Revert "Limpeza de pycache e atualizaçã...
Caetano Petry • 3 days ago

Limpeza de pycache e atualização do .gi...
Caetano Petry • 3 days ago

Merge branch 'antes-da-api-asaas' of ht...
Caetano Petry • 3 days ago

Revert "Limpeza de pycache e atualizaçã...
Caetano Petry • 3 days ago

Revert "Limpeza de pycache e atualizaçã...
Caetano Petry • 3 days ago

Limpeza de pycache e atualização do .gi...
Caetano Petry • 3 days ago

Revert "Limpeza de pycache e atualizaçã...
Caetano Petry • 3 days ago

Limpeza de pycache e atualização do .gi...
Caetano Petry • 3 days ago

Merge branch 'main' of https://github.c...
Pietro Tamanini, Caetano Petry • 4 da...

Update produtos.html
Caetano Petry • 3 days ago

Alterações em finalizar carrinho, correçã...
Caetano Petry, Caetano Petry • 4 days...

Adição de novos integrantes á página so...
Caetano Petry, Caetano Petry • 3 days...

Adição de nova página de funcionários, ...
Caetano Petry, Caetano Petry • 4 days...

Adição de novos integrantes á página so...

Add admin documentação

Adição página "Documentação" no painel

Djonathan Tamanini ef83c43

4 changed files

routes\admin_routes.py
static\docs\pdf\D...ra-de-Negocio.pdf
view\admin\base.html
view\admin\documentation.html

9 - Deploy:

Onde o Sistema está hospedado ?

‘http://gamechip.tamanini.dev.br’ - GameChip

9.1 - Como rodar o projeto ?

Backend - VsCode: app.py python

Frontend - http://localhost:5000

Banco de Dados - mysql -u root -p < banco.sql

10 - Considerações Finais:

O que aprendemos ?

‘Neste projeto final aprendemos a nos desenvolver em equipe, trabalhando juntos e superando os desafios dos códigos, juntamente com pessoas diferentes, que vieram para nos ajudar a pensar de forma mais limpa e organizada.’

Melhorias futuras ?

‘Ainda não paramos para pensar em melhorias futuras, mas se for para melhorar alguma coisa vai ser organizacao antes de codificar.’

O que ainda pode ser implementado ?

‘Ainda estamos querendo adicionar uma verificação no Email após a compra do produto’

Muito Obrigado!

- **GameChip**